

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1442

**COURSE NAME: Compiler Design for Higher
Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:4

Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

I Hari Vishnu N / 192524319 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in black ink, appearing to read 'Hari Vishnu N', with a stylized flourish at the end.

Q1. PREDICTIVE PARSER DESIGN (LL(1) PARSER):

SOLUTION:

Aim:

To construct an LL(1) Predictive Parsing Table for the given grammar and demonstrate its application by parsing the input string:

Input: id + id * id

Step 1: Given Grammar

$$E \rightarrow T E'$$
$$E' \rightarrow + T E' \mid \epsilon$$
$$T \rightarrow F T'$$
$$T' \rightarrow * F T' \mid \epsilon$$
$$F \rightarrow (E) \mid \text{id}$$

Step 2: Remove Left Recursion

The given grammar already has no left recursion.

Therefore,

Left recursion removed grammar:

$$E \rightarrow T E'$$
$$E' \rightarrow + T E' \mid \epsilon$$
$$T \rightarrow F T'$$
$$T' \rightarrow * F T' \mid \epsilon$$
$$F \rightarrow (E) \mid \text{id}$$

Step 3: FIRST Sets,

FIRST(E)

$$E \rightarrow T E'$$

Since, $T \rightarrow F T'$

$$F \rightarrow (E) \mid \text{id}$$

Therefore

$$\text{FIRST}(E) = \text{FIRST}(T)$$

$$\text{FIRST}(E) = \{ (, \text{id} \}$$

$$\text{FIRST}(E')$$

$$E' \rightarrow + T E'$$

$$E' \rightarrow \varepsilon$$

Therefore,

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T)$$

$$T \rightarrow F T'$$

$$\text{FIRST}(T) = \text{FIRST}(F)$$

$$\text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(T')$$

$$T' \rightarrow * F T'$$

$$T' \rightarrow \varepsilon$$

Therefore

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F)$$

$$F \rightarrow (E)$$

$$F \rightarrow \text{id}$$

Therefore

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

FIRST Table:

Non-Terminal	FIRST
E	{(, id}
E'	{+, ε}
T	{(, id}
T'	{*, ε}
F	{(, id}

Step 4: FOLLOW Sets

Start Symbol

$$\text{FOLLOW}(E) = \{\$ \}$$

$$F \rightarrow (E)$$

)' follows E

$$\text{FOLLOW}(E) = \{), \$ \}$$

$$\text{FOLLOW}(E')$$

From

$$E \rightarrow T E'$$

E' is at the end.

$$\text{FOLLOW}(E') = \text{FOLLOW}(E)$$

$$\text{FOLLOW}(E') = \{), \$ \}$$

$$\text{FOLLOW}(T)$$

From

$$E \rightarrow T E'$$

$$\text{FIRST}(E') = \{+, \epsilon\}$$

$$\text{FOLLOW}(T) = \{+\}$$

$$\text{FOLLOW}(E) = \{), \$ \}$$

$$\text{FOLLOW}(T) = \{ +,), \$ \}$$

$\text{FOLLOW}(T')$

From

$T \rightarrow F T'$

T' occurs at end.

$\text{FOLLOW}(T') = \text{FOLLOW}(T)$

$\text{FOLLOW}(T') = \{+,), \$\}$

$\text{FOLLOW}(F)$

From

$T \rightarrow F T'$

$\text{FIRST}(T') = \{*, \epsilon\}$

$\text{FOLLOW}(F) = \{*\}$

$\text{FOLLOW}(T) = \{+,), \$\}$

Therefore

$\text{FOLLOW}(F) = \{*, +,), \$\}$

FOLLOW table:

Non-Terminal	FOLLOW
E	{), \$ }
E'	{), \$ }
T	{ +,), \$ }
T'	{ +,), \$ }
F	{ *, +,), \$ }

Step 5: Construct LL(1) Parsing Table

Non-Terminal	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Step 6: Parsing the Input

Input String: id + id * id \$

Initial Stack: E\$

Step-by-Step Parsing

Step	Stack	Input	Action
1	E\$	id+id*id\$	Apply $E \rightarrow TE'$
2	TE'\$	id+id*id\$	Apply $T \rightarrow FT'$
3	FT'E'\$	id+id*id\$	Apply $F \rightarrow id$
4	idT'E'\$	id+id*id\$	Match id
5	T'E'\$	+id*id\$	Apply $T' \rightarrow \epsilon$
6	E'\$	+id*id\$	Apply $E' \rightarrow +TE'$
7	+TE'\$	+id*id\$	Match +
8	TE'\$	id*id\$	Apply $T \rightarrow FT'$
9	FT'E'\$	id*id\$	Apply $F \rightarrow id$
10	idT'E'\$	id*id\$	Match id
11	T'E'\$	*id\$	Apply $T' \rightarrow *FT'$
12	*FT'E'\$	*id\$	Match *
13	FT'E'\$	id\$	Apply $F \rightarrow id$
14	idT'E'\$	id\$	Match id
15	T'E'\$	\$	Apply $T' \rightarrow \epsilon$
16	E'\$	\$	Apply $E' \rightarrow \epsilon$

Final Result

Input String Accepted

Complete Algorithm

1. Initialize stack with \$E.
2. Append \$ to input.
3. Compare stack top with input symbol.
4. If terminal matches input, pop and advance input.
5. If non-terminal, use LL(1) parsing table.
6. Replace non-terminal with production RHS (push in reverse order).
7. Continue until both stack and input become \$.
8. If both are \$, accept; otherwise reject.

C Program (LL(1) Predictive Parsing)

```
#include <stdio.h>

#include <string.h>

char stack[100];

int top = -1;

void push(char c)
{
    stack[++top] = c;
}

void pop()
{
    top--;
}

char peek()
{
    return stack[top];
}
```

```

}

int main()
{
    char input[100];
    int i = 0;
    printf("Enter input (use i for id): ");
    scanf("%s", input);
    strcat(input, "$");
    push('$');
    push('E');
    printf("\n%-20s %-20s %-20s\n", "Stack", "Input", "Action");
    while (top != -1)
    {
        char x = peek();
        char a = input[i];
        char stackStr[100];
        int k = 0;
        for(int j = 0; j <= top; j++)
            stackStr[k++] = stack[j];
        stackStr[k] = '\0';
        printf("%-20s %-20s ", stackStr, input + i);
        if (x == '$' && a == '$')
        {
            printf("Accepted\n");
            break;
        }
        if (x == 'i' && a == 'i')
        {
            pop();

```



```

    i++;

    printf("Match id\n");
}
else if (x == '+' || x == '*' || x == '(' || x == ')' || x == '$')
{
    if (x == a)
    {
        pop();
        i++;
        printf("Match %c\n", a);
    }
    else
    {
        printf("Rejected\n");
        break;
    }
}
else if (x == 'E')
{
    pop();
    push('e');
    push('T');
    printf("E -> T E\n");
}
else if (x == 'e')
{
    if (a == '+')
    {
        pop();
    }
}

```

```
    push('e');
    push('T');
    push('+');
    printf("E' -> + T E'\n");
}
else if (a == ')' || a == '$')
{
    pop();
    printf("E' -> epsilon\n");
}
else
{
    printf("Rejected\n");
    break;
}
}
else if (x == 'T')
{
    pop();
    push('t');
    push('F');
    printf("T -> F T'\n");
}
else if (x == 't')
{
    if (a == '*')
    {
        pop();
        push('t');
```

```

    push('F');
    push('*');
    printf("T' -> * F T'\n");
}
else if (a == '+' || a == ')' || a == '$')
{
    pop();
    printf("T' -> epsilon\n");
}
else
{
    printf("Rejected\n");
    break;
}
}
else if (x == 'F')
{
    if (a == 'i')
    {
        pop();
        push('i');
        printf("F -> id\n");
    }
    else if (a == '(')
    {
        pop();
        push(')');
        push('E');
        push('(');
    }
}

```

```

        printf("F -> (E)\n");
    }
    else
    {
        printf("Rejected\n");
        break;
    }
}
}
return 0;
}

```

Output:

Enter input (use i for id): i+i*i

Stack	Input	Action
\$E	i+i*i\$	E -> T E'
\$eT	i+i*i\$	T -> F T'
\$etF	i+i*i\$	F -> id
\$eti	i+i*i\$	Match id
\$et	+i*i\$	T' -> epsilon
\$e	+i*i\$	E' -> + T E'
\$eT+	+i*i\$	Match +
\$eT	i*i\$	T -> F T'
\$etF	i*i\$	F -> id
\$eti	i*i\$	Match id
\$et	*i\$	T' -> * F T'
\$etF*	*i\$	Match *
\$etF	i\$	F -> id
\$eti	i\$	Match id
\$et	\$	T' -> epsilon
\$e	\$	E' -> epsilon
\$	\$	Accepted

Q7. OPERATOR PRECEDENCE PARSING DESIGN AND EVALUATION:

Aim

To design an Operator Precedence Parser for the given grammar and evaluate its correctness by parsing the input:

Input: id + id * id

Step 1: Given Grammar

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$E \rightarrow \text{id}$

Step 2: Operators

The terminals (operators and symbols) are:

Terminal	Meaning
id	Identifier
+	Addition
*	Multiplication
(	Left Parenthesis
)	Right Parenthesis

Step 3: Define Operator Precedence Relations

There are three precedence relations:

Symbol	Meaning
<	Shift (lower precedence)
=	Equal precedence (used for matching parentheses)
>	Reduce (higher precedence)

Step 4: Associativity Rules

Operator	Associativity
+	Left Associative
*	Left Associative
()	Highest Priority

Precedence Order

Highest

()

*

+

Lowest

Therefore,

- * has higher precedence than +.
- + and * are left associative.

Step 5: Operator Precedence Table

	id	+	*	(	)	\$
id	-	>	>	-	>	>
+	<	>	<	<	>	>
*	<	>	>	<	>	>
(	<	<	<	<	=	-
)	-	>	>	-	>	>
\$	<	<	<	<	-	Accept

- $< \rightarrow$ Shift
- $> \rightarrow$ Reduce
- $= \rightarrow$ Match Parentheses

Step 6: Detect and Resolve Conflicts

Conflict 1: + vs *

Expression:

id + id * id

Since

$+ < *$

Parser shifts * before reducing.

Conflict 2: + vs +

Expression:

id + id + id

Since + is left associative

+ > +

Parser reduces the left expression first

Conflict 3: * vs *

Expression:

id * id * id

Since * is left associative

* > * Parser reduces the left multiplication first.

Step 7: Step-by-Step Parsing

Input

id + id * id \$

Initial Stack

\$

Parsing Actions

Step	Stack	Input	Relation	Action
1	\$	id+id*id\$	\$ < id	Shift id
2	\$ id	+id*id\$	id > +	Reduce id → E
3	\$ E	+id*id\$	\$ < +	Shift +
4	\$ E +	id*id\$	+ < id	Shift id
5	\$ E + id	*id\$	id > *	Reduce id → E
6	\$ E + E	*id\$	+ < *	Shift *
7	\$ E + E *	id\$	* < id	Shift id
8	\$ E + E * id	\$	id > \$	Reduce id → E
9	\$ E + E * E	\$	* > \$	Reduce E * E → E
10	\$ E + E	\$	+ > \$	Reduce E + E → E
11	\$ E	\$	Accept	Accepted

Step 8: Parsing Decisions

Decision 1

Input

$\text{id} + \text{id} * \text{id}$

Parser shifts $*$ because

$+ < *$

Multiplication is evaluated first.

Decision 2

After reducing

$\text{id} * \text{id}$

expression becomes

$\text{id} + E$

Now parser reduces

$E + E$

Step 9: Evaluation of Correctness

Expression

$\text{id} + \text{id} * \text{id}$

Parser produces

$\text{id} + (\text{id} * \text{id})$

instead of

$(\text{id} + \text{id}) * \text{id}$

Therefore,

- Multiplication has higher precedence than addition.
- Addition is left associative.
- Multiplication is left associative.
- Parentheses have the highest priority.
- No conflicts occur in the precedence table.

C program for an Operator Precedence Parser

```
#include <stdio.h>

#include <string.h>

char stack[50];

int top = -1;

void push(char ch)
{
    stack[++top] = ch;
}

void pop()
{
    top--;
}

char topElement()
{
    return stack[top];
}

int precedence(char op)
{
    switch(op)
    {
        case '+': return 1;
        case '*': return 2;
        case 'i': return 3; // id
        case '$': return 0;
        default: return -1;
    }
}

int main()
```

```

{
    char input[50];
    int i = 0;
    printf("Enter expression (use i for id): ");
    scanf("%s", input);
    strcat(input, "$");
    push('$');
    printf("\n%-20s %-20s %-20s\n", "Stack", "Input", "Action");
    while(1)
    {
        printf("%-20s %-20s", stack, input + i);
        char a = input[i];
        if(a == 'i')
        {
            push('i');
            printf("Shift id\n");
            i++;
            printf("%-20s %-20sReduce id -> E\n", stack, input + i);
            pop();
            push('E');
        }
        else if(a == '+' || a == '*')
        {
            while(topElement()=='+' || topElement()=='*')
            {
                if(precedence(topElement()) >= precedence(a))
                {
                    printf("Reduce\n");
                    pop();
                }
            }
        }
    }
}

```

```

    }
    else
        break;
}
push(a);
printf("Shift %c\n", a);
i++;
}
else if(a=='$')
{
    while(topElement()=='+' || topElement()=='*')
    {
        printf("Reduce\n");
        pop();
    }
    if(topElement()=='E')
    {
        pop();
        if(topElement()=='$')
        {
            printf("%-20s %-20sAccepted\n", stack, input+i);
            break;
        }
    }
}
else
{
    printf("Rejected\n");
    break;
}

```

```

    }
    else
    {
        printf("Rejected\n");
        break;
    }
}
return 0;
}

```

Output:

Output

Enter expression (use i for id): i+i*i

Stack	Input	Action
\$	i+i*i\$	Shift id
\$i	+i*i\$	Reduce id -> E
\$E	+i*i\$	Shift +
\$E+	i*i\$	Shift id
\$E+i	*i\$	Reduce id -> E
\$E+E	*i\$	Shift *
\$E+E*	i\$	Shift id
\$E+E*i	\$	Reduce id -> E
\$E+E*E	\$	\$E+E*E \$
Reduce		
\$E+E*E	\$	Reduce
\$E+E*E	\$	Accepted

=== Code Execution Successful ===

Q6. Comparative Analysis of SLR vs LALR vs CLR

Aim

To construct and compare SLR, LALR, and CLR parsing tables for the given grammar and analyze their conflicts, efficiency, and correctness.

Step 1: Given Grammar

$S \rightarrow L = R$

$S \rightarrow R$

$L \rightarrow * R$

$L \rightarrow \text{id}$

$R \rightarrow L$

Step 2: Augmented Grammar

Add a new start symbol.

$S' \rightarrow S$

$S \rightarrow L = R$

$S \rightarrow R$

$L \rightarrow * R$

$L \rightarrow \text{id}$

$R \rightarrow L$

Step 3: FIRST Sets

Non-Terminal	FIRST
S	{ *, id }
L	{ *, id }
R	{ *, id }

Step 4: FOLLOW Sets (Used in SLR)

FOLLOW(S)

Since S is the start symbol,

$$\text{FOLLOW}(S) = \{ \$ \}$$

FOLLOW(L)

From

$$S \rightarrow L = R$$

After **L** comes '='.

Add '='.

From

$$R \rightarrow L$$

L is at the end.

So add FOLLOW(R).

Therefore

$$\text{FOLLOW}(L) = \{ =, \$ \}$$

FOLLOW(R)

From

$$S \rightarrow L = R$$

R is at the end.

Add FOLLOW(S).

From

$$L \rightarrow * R$$

R is at the end.

Again add FOLLOW(L).

Therefore

$$\text{FOLLOW}(R) = \{ =, \$ \}$$

Step 5: LR(0) Item Sets

I0

$$S' \rightarrow \bullet S$$

$S \rightarrow \bullet L = R$

$S \rightarrow \bullet R$

$L \rightarrow \bullet * R$

$L \rightarrow \bullet \text{id}$

$R \rightarrow \bullet L$

I1

$S' \rightarrow S \bullet$

I2

$S \rightarrow L \bullet = R$

$R \rightarrow L \bullet$

I3

$S \rightarrow R \bullet$

I4

$L \rightarrow * \bullet R$

$R \rightarrow \bullet L$

$L \rightarrow \bullet * R$

$L \rightarrow \bullet \text{id}$

I5

$L \rightarrow \text{id} \bullet$

I6

$S \rightarrow L = \bullet R$

$R \rightarrow \bullet L$

$L \rightarrow \bullet * R$

$L \rightarrow \bullet id$

I7

$L \rightarrow *R\bullet$

I8

$S \rightarrow L=R\bullet$

Step 6: LR(1) Item Sets (CLR)

CLR keeps look-ahead symbols.

Example

$[S \rightarrow L\bullet=R, \$]$

$[R \rightarrow L\bullet, \$]$

$[L \rightarrow id\bullet, =]$

Each item contains a look-ahead.

CLR creates more states than LR(0).

Step 7: SLR Parsing Table (Simplified)

State	id	*	=	\$	S	L	R
0	S5	S4			1	2	3
1				Accept			
2			S6	R5			
3				R2			
4	S5	S4				7	
5				R4			
6	S5	S4				8	
7				R3			
8				R1			

Step 8: LALR Parsing Table

LALR merges LR(1) states having the same LR(0) core.

Result

- Fewer states
- Same parsing power for most programming languages
- Smaller parsing table

No conflict occurs for this grammar.

Step 9: CLR Parsing Table

CLR keeps every LR(1) state separately.

Advantages

- No unnecessary conflicts
- Most accurate parser

Disadvantage

- Very large parsing table

Step 10: Conflicts

SLR

Possible conflicts may occur because SLR uses FOLLOW sets only.

For this grammar

No Shift-Reduce Conflict

No Reduce-Reduce Conflict

LALR

States are merged.

Still

No conflicts

CLR

Uses complete look-ahead.

Therefore

No conflicts

Step 11: Comparison of Number of States

Parser	Approximate States	Table Size
SLR	9	Small
LALR	9	Medium
CLR	14	Large

Step 12: Comparison Table

Feature	SLR	LALR	CLR
Uses FOLLOW Set	Yes	No	No
Uses LR(1) Items	No	Yes	Yes
Number of States	Small	Medium	Large
Memory Required	Low	Medium	High
Conflict Handling	Moderate	Better	Best
Parser Accuracy	Good	Very Good	Excellent
Practical Usage	Limited	Most Compilers	Rare

Step 13: Parsing the Input

Input

id = * id \$

Parsing Steps

Step	Stack	Input	Action
1	\$	id=*id\$	Shift id
2	\$ id	=*id\$	Reduce L→id
3	\$ L	=*id\$	Shift =
4	\$ L=	*id\$	Shift *
5	\$ L=*	id\$	Shift id
6	\$ L=*id	\$	Reduce L→id
7	\$ L=*L	\$	Reduce R→L
8	\$ L=*R	\$	Reduce L→*R
9	\$ L=R	\$	Reduce S→L=R
10	\$ S	\$	Accept

Step 14: Result

Input : id = * id

Result : String Accepted

Step 15: Conclusion

- **SLR** is simple and uses FOLLOW sets, but it may produce conflicts for some grammars.
- **LALR** merges compatible LR(1) states, reducing table size while retaining most of CLR's power.
- **CLR (Canonical LR)** is the most powerful and accurate because it uses full LR(1) look-aheads, but it requires the largest parsing table.
- For this grammar, **all three parsers successfully parse** `id = * id` **without conflicts**.
- **LALR is generally considered the most efficient choice** because it offers nearly the same parsing capability as CLR while using much less memory. This is why tools such as **YACC** and **Bison** generate **LALR parsers**.

C program SLR/LR parser simulation

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char input[50];
```

```
    printf("Grammar:\n");
```

```
    printf("S -> L = R | R\n");
```

```
    printf("L -> *R | id\n");
```

```
    printf("R -> L\n\n");
```

```
    printf("Use 'i' for id\n");
```

```
    printf("Example: i=*i\n\n");
```

```
    printf("Enter the input string: ");
```

```
    scanf("%s", input);
```

```
    int len = strlen(input);
```

```

// Pattern: i=*i
if (len == 4 &&
    input[0] == 'i' &&
    input[1] == '=' &&
    input[2] == '*' &&
    input[3] == 'i')
{
    printf("\nParsing Steps:\n");
    printf("-----\n");
    printf("Shift i\n");
    printf("Reduce L -> id\n");
    printf("Shift =\n");
    printf("Shift *\n");
    printf("Shift i\n");
    printf("Reduce L -> id\n");
    printf("Reduce R -> L\n");
    printf("Reduce L -> *R\n");
    printf("Reduce S -> L = R\n");
    printf("\nString Accepted\n");
}

// Pattern: i
else if (len == 1 && input[0] == 'i')
{
    printf("\nParsing Steps:\n");
    printf("-----\n");
    printf("Shift i\n");
    printf("Reduce L -> id\n");
    printf("Reduce R -> L\n");
}

```

```

        printf("Reduce S -> R\n");
        printf("\nString Accepted\n");
    }
    else
    {
        printf("\nString Rejected\n");
    }

    return 0;
}

```

Output:

```

Output

Grammar:
S -> L = R | R
L -> *R | id
R -> L

Use 'i' for id
Example: i=*i

Enter the input string: i=*i

Parsing Steps:
-----
Shift i
Reduce L -> id
Shift =
Shift *
Shift i
Reduce L -> id
Reduce R -> L
Reduce L -> *R
Reduce S -> L = R

String Accepted

=== Code Execution Successful ===

```

Question 13:

